I am providing:

1. Complete TechAdmin source code ZIP
2. SolarWinds integration notebook (.ipynb)
3. Existing database models
4. Existing agent implementations
5. Existing ticket generation framework
6. Existing UI and API implementations

Your task is to thoroughly analyze the entire codebase and design and implement a new production-grade:

SolarWinds AI Operations Agent

Do NOT assume any folder structure.

Do NOT create arbitrary folders.

First understand:

Current project architecture
Agent implementation patterns
Database patterns
Repository patterns
Service patterns
API patterns
Scheduler patterns
Ticketing patterns
Dashboard patterns
Logging patterns
Authentication patterns
Guardrail patterns

Then design and implement the solution according to the architecture already established within TechAdmin.

The goal is to make the new SolarWinds Agent feel like a native component of the system rather than a separately developed add-on.

Business Goal

Build an AI-powered SolarWinds Operations capability that allows TechAdmin to:

Collect SolarWinds telemetry
Store historical snapshots
Generate health scores
Perform trend analysis
Generate AI reports
Forecast future risks
Create tickets automatically
Provide root-cause style insights
Support natural language querying through an AI agent

Existing SolarWinds Data

The current SolarWinds integration already retrieves:

NodeID
Caption
IPAddress
Status
StatusDescription
CPULoad
PercentMemoryUsed
ResponseTime
AvgResponseTime
Vendor
MachineType
LastSync
SystemUpTime
NodeDescription
City
ITSM_Location
NodeCategory

NodeType
Contact

Current environment contains approximately:

1572 monitored devices

Current categories include:

Server
Network
Telecom
Cloud
Collaboration
Client IP
URL

Architecture Analysis Phase

Before implementing anything:

Perform a complete architecture review and document:

Current project layers
Existing agent architecture
Existing patterns being followed
Database design conventions
Migration mechanisms
Repository patterns
Service patterns
API structure
Authentication mechanisms
Authorization mechanisms
Scheduling framework
Logging standards
Error handling standards
Dashboard framework
Ticket generation architecture

Provide a gap analysis before implementation.

Do not duplicate existing functionality.

Reuse existing components whenever possible.

Database Design Requirements

Design database tables according to the conventions already established in TechAdmin.

Provide:

ORM models
Migration scripts
Indexes
Relationships
Repository classes
CRUD operations

The solution should support:

Master Device Inventory

Store:

NodeID
Caption
IPAddress
Vendor
MachineType
NodeCategory
NodeType
City
ITSM_Location
Contact
NodeDescription

Historical Snapshots

Store every collection cycle:

Collection Timestamp
NodeID
Status
StatusDescription
CPULoad
PercentMemoryUsed
ResponseTime
AvgResponseTime
HealthCurrent
SystemUptime
LastSync

Historical records must never be overwritten.

The goal is to build forecasting and trend analysis from historical telemetry.

Daily Summary Storage

Persist daily aggregated results such as:

Total Devices
High Health Devices
Medium Health Devices
Low Health Devices
Average CPU
Average Memory
Average Response Time
Category Breakdown
Vendor Breakdown

This will support executive dashboards and faster report generation.

AI Insights Storage

Store generated AI findings:

Risk Score
Risk Level
Root Cause
Recommendations
Executive Summary
Analysis Metadata

Forecast Storage

Store generated forecasts:

Predicted CPU
Predicted Memory
Predicted Health
Confidence Score
Risk Indicators
Forecast Rationale

Ticket Tracking Storage

Store ticket mappings:

Node
Ticket
Reason
Risk Level
Status
Audit Information

Health Classification Engine

Implement a reusable health scoring engine.

Rules:

LOW

If:

Status = Down
Status = Critical
Status = Unreachable

OR

CPU > 90%

OR

Memory > 90%

MEDIUM

If Low does not apply and:

CPU >= 75%

OR

Memory >= 80%

OR status:

Warning
Shutdown
Unmanaged
Unknown
Inactive

Undefined

HIGH

If:

Status = Up

AND

CPU < 75%

AND

Memory < 80%

Missing CPU or Memory values must not downgrade an Up node.

Data Collection Architecture

Design a complete ingestion workflow.

Requirements:

Daily automated execution
Target schedule:
8:00 AM IST

The workflow should:

Connect to SolarWinds
Fetch telemetry
Validate results
Calculate health
Persist snapshots
Generate summaries
Generate insights
Generate forecasts

Create tickets if required

Analyze existing TechAdmin scheduling capabilities first.

If no scheduler exists, propose the most appropriate implementation pattern.

Reporting Requirements

Generate:

Daily Health Report

Example:

1572 devices monitored
1320 High Health
185 Medium Health
67 Low Health
Average CPU: 42%
Average Memory: 65%
12 new unhealthy devices detected

Executive Summary

Example:

Network category contains the highest number of degraded devices.
Medium Health devices increased by 8% compared to yesterday.
Most incidents are driven by CPU utilization.

Operational Recommendations

Example:

Investigate memory-intensive workloads.

Review long-running services.
Evaluate storage growth trends.

Forecasting Requirements

Use historical telemetry stored in TechAdmin.

Support:

7-day forecast
30-day forecast
90-day forecast

Forecast:

CPU Utilization
Memory Utilization
Health Degradation
Response Time Changes

Generate:

Risk Indicators
Confidence Scores
Forecast Explanations

The design should support future migration to advanced forecasting models.

Ticket Automation

Analyze and reuse the existing ticket generation implementation present in TechAdmin.

Do not create another independent ticketing framework.

Generate tickets when:

HealthCurrent = Low

or

Forecast identifies significant future risk

Ticket payload should include:

NodeID
Device Name
IP Address
Category
Status
CPU
Memory
Health
Risk
Recommendations
Root Cause

Implement ticket deduplication.

Avoid multiple open tickets for the same persistent issue.

AI Agent Capabilities

The SolarWinds Agent should support:

Health Queries

Show all unhealthy devices
Show critical network devices
Show devices with CPU above 90%
Show devices with memory above 80%

Trend Analysis

Which devices degraded in the last 30 days

Health trend for a device
Category health trend
Most unstable devices

Forecast Queries

Devices likely to become unhealthy
CPU growth risks
Memory exhaustion risks
Upcoming operational concerns

Ticket Queries

Show generated tickets
Why was a ticket generated
Open ticket summary
Devices without tickets

Executive Queries

Generate today's SolarWinds report
Generate weekly report
Top 20 risks
Overall infrastructure health summary

Dashboard Requirements

Design dashboard enhancements that align with existing TechAdmin UI standards.

Support:

Total Devices
High Health

Medium Health
Low Health
Open Tickets
Forecasted Risks

Visualizations:

Category Distribution
Vendor Distribution
Health Trends
CPU Trends
Memory Trends
Forecast Trends
Ticket Trends

Tables:

Critical Devices
Top Risk Devices
Forecasted Risks
Open Tickets

Development Requirements

All generated code must include:

Docstrings
Type Hints
Structured Logging
Error Handling
Comments
Architecture Notes
Dependency Documentation

Every file should clearly explain:

Why it exists
What business function it serves

How it integrates with TechAdmin
How it interacts with SolarWinds
How it participates in the data flow

Deliverables

Provide:

1. Current-state architecture assessment.
2. Gap analysis.
3. Proposed architecture.
4. Database schema design.
5. ORM models.
6. Migration scripts.
7. Repository implementation.
8. Service implementation.
9. Ingestion workflow.
10. Health scoring engine.
11. AI reporting engine.
12. Forecasting engine.
13. Ticket integration.
14. Agent implementation.
15. API endpoints.
16. Dashboard implementation.
17. Test strategy.
18. Deployment strategy.
19. Operational runbook.
20. File-by-file explanation of every created or modified component.

Before making any implementation decisions, fully analyze the uploaded TechAdmin ZIP and notebook and adapt all design decisions to the conventions already present within the project.